Hướng dẫn triển khai ACT nhận biết ý định (intent-aware ACT) cho HRI: taxonomy → biểu diễn → chèn vào ACT → component dropout → đánh giá → predictor

TL;DR

- Kế hoạch của bạn khả thi về kỹ thuật và đã có tiền lệ khá gần ở cấp code: cách đơn giản nhất là thêm token ý định vào sau cặp [latent, proprio] trong `transformer.py` của ACT, mở rộng `additional_pos_embed`, và đưa cùng các token đó vào CVAE encoder. Repo P+ACT (Likerener/prox_learning) đã sửa đúng các vị trí này để thêm token proximity, còn MT-ACT (RoboAgent) cho thấy FiLM tốt hơn ghép token 5–10%. Vì vậy hãy làm token trước, FiLM (khởi tạo identity) làm phương án dự phòng.
- Schema What-target/What-action/When/How nên bám theo Ego4D STA (noun + verb + box + time-to-contact, ngưỡng TTC 0,25 s) và các mốc PRE/CONTACT/PNR/POST của Ego4D, cộng với hai pha pre-handover/physical handover của Ortenzi et al. Từ đó định nghĩa t_onset, t_clear, t_event theo cách thao tác được và tính τ, ξ trực tiếp từ keypoint đã ghi.
- Rủi ro lớn nhất không nằm ở kiến trúc mà ở *rò rỉ* (leakage): (i) intent trùng với hành động robot (đặc biệt trong Instructor); (ii) "action leakage" của action chunking làm robot đi trước (false start), như Xu et al. 2026 đã chỉ ra; (iii) oracle τ/ξ chỉ hợp lệ khi chuyển động người được phát lại (replay) và không phản ứng với robot. Vì vậy cần thiết kế manipulation check và đo false start ngay từ đầu, rồi chấm điểm bằng các metric tương tác kiểu HRIBench (CSR, Rsp, TSync, CFR).

Key Findings

1. **HRIBench (arXiv 2607.13056, Liu, Zhang, Zhang, Wang, Zhou, Jin; Renmin Univ.; preprint 05/07/2026)** dùng đúng ba vai Instructor/Collaborator/Intruder và đã đánh giá ACT. Benchmark gồm 13 task, hơn 50 biến thể quỹ đạo mỗi task, hơn 650 episode. Mỗi "scenario script" là chuỗi $a_t = (r_t, g_t, c_t, m_t, \rho_t)$, lần lượt là vai, mục tiêu ngữ nghĩa, ràng buộc thời gian/nhân quả, phân phối chuyển động người, và điều kiện thưởng/thành công. CSR tốt nhất là 0,533 (Instructor), 0,500 (Collaborator) và chỉ 0,100 (Intruder). arxiv Đây là chuẩn tham chiếu tự nhiên cho đồ án của bạn.
2. **Learning to Assist (arXiv 2606.12475, Xu, Li, Cuellar, Hagenow; UW–Madison & MIT; preprint 10/06/2026)** mô hình hóa đúng $\pi(a \mid o, l, z_h)$ với z_h là ý định cộng tác tiềm ẩn của người. Nhóm này phát hiện "demonstration action leakage": action chunk vắt qua thời điểm chuyển ý định gây hỗ trợ sớm, và lỗi tăng khi execution horizon dài hơn. arxiv +2 Đây chính là cơ chế bạn cần kiểm soát trong thiết kế 2×2 "early response".
3. **ACT gốc (tonyzhaozh/act)**: các mặc định đã kiểm chứng từ README/ `imitate_episodes.py` là chunk_size 100, kl_weight 10, hidden_dim 512, dim_feedforward 3200, enc_layers 4, dec_layers 7, nheads 8, lr 1e-5, lr_backbone 1e-5, batch 8, ResNet18. github github **MT-ACT** dùng chunk_size 20 và batch 64, và theo ablation của chính bài báo, chunk 20 tốt nhất. github arxiv
4. **Conditioning dropout**: Octo "randomly zero out" language hoặc goal image cho mỗi mẫu, và token không tồn tại bị mask hoàn toàn. arxiv arxiv BESO (Reuss, Li, Jia,

Lioutikov; RSS 2023; arXiv 2304.02532) "applying a dropout rate of 0.1 to the goal g, which also trains an implicit goal-independent policy π(a|s)". Các mô hình CFG thường thay điều kiện bằng *learned null embedding*. (arxiv) Như vậy, việc dùng token "unknown" riêng cho từng thành phần đã có tiền lệ.

5. **Predictor nhẹ có sẵn**: siMLPe (Guo et al., WACV 2023, arXiv 2207.01567; DCT + MLP, chỉ 0,14M tham số, 0,138M theo Bảng 5) cho ξ; ManiCast (CoRL 2023) cho forecast "cost-aware" (pretrain AMASS, fine-tune trên CoMaD thu bằng motion capture); InteRACT (ICRA 2024) cho dự đoán ý định có điều kiện theo hành động robot. (arxiv) (github)

Details

Bước 0 — Định vị đồ án so với tài liệu gần nhất

- **HRIBench** tạo chuyển động người bằng HY-Motion-1.0, sau đó tinh chỉnh với ràng buộc gán nhãn trên tay, hướng thân và hướng tiếp cận. Trong 2.000 quỹ đạo sinh ra, 786 qua được bộ lọc tự động và kiểm tra thủ công. Mỗi task dùng 50 quỹ đạo để thích nghi và giữ lại 10 để đánh giá, trên Franka Panda. Với Intruder, "perturbation schedules are generated independently of the evaluated policy and kept fixed across models": người không phản ứng với robot. (arxiv) Đây chính là chế độ mà oracle τ/ξ của bạn hợp lệ. Nghiên cứu real-world của họ dùng LeRobot SO-100 với 20 quỹ đạo teleop mỗi task, và GR00T N1.5 tăng từ 0,10 lên 0,43 khi thêm dữ liệu sim. (arxiv)
- **Metric HRIBench** nên dùng lại: Effectiveness (CSR, CT, Idle Ratio), Coordination (TSync, Response Latency, Order Compliance), Safety (CFR, Human Contact Safety, Contradictory Instruction Recognition, Disruption Success Rate). Phát hiện đáng chú ý: với Intruder, OC gần hoàn hảo nhưng CSR rất thấp. (arxiv) Nghĩa là policy tuân thủ thứ tự danh nghĩa nhưng không biết nhường hoặc phục hồi, và đây là chỗ intent "reach into / approach" có thể tạo khác biệt.
- **Learning to Assist**: UFactory X-Arm 6, hai RealSense D435i (scene + wrist), demo do một người ngồi cộng tác và người thứ hai teleop bằng SpaceMouse. Có ba task: Abstract Assembly, Mini-Chair (H2R/R2H handover) và Radio (demo trung bình 5,5 phút). Họ giảm false start bằng inference-time steering với "basin points" sinh tự động, và user study 16 người cho thấy hợp tác nhanh hơn, ít lỗi hơn so với baseline horizon ngắn. (arxiv) (arxiv) Bài học cho bạn: khi thêm oracle τ, policy có thể học "đếm ngược", nhưng nếu không có τ thì chunk dài dễ gây false start. Vì vậy *false-start rate* phải là metric chính trong so sánh early vs. clear response.

Bước 1 — Taxonomy và pipeline gán nhãn

1.1. Mượn định nghĩa từ benchmark anticipation

Nguồn	Nhãn	Thời gian	Metric
Ego4D STA	noun (v1: 87 lớp, v2: 128), verb (v1: 74, v2: 81), bbox của next-active object, TTC	TTC = thời gian từ khung quan sát đến khung contact	Top-5 mAP: Noun, Noun+Verb, Noun+TTC, Overall; khớp khi IoU > 0,5 và sai số TTC < 0,25 s
Ego4D FHO critical frames	PRE, CONTACT, PNR, POST cho mỗi hành động được thuyết minh	Hand box tại contact, pre-condition và 0,5/1/1,5 s trước pre-condition	—
EPIC-KITCHENS-100	97 verb, 300 noun (cặp verb–noun = action)	anticipation time τ_a = 1 s trước khi action bắt đầu	Mean Top-5 Recall (class-mean)
RU-LSTM (Furnari & Farinella)	verb/noun	"time to action" TtA(k): thời điểm sớm nhất dự đoán đúng theo top-k	mTtA(k)
Ortenzi et al., IEEE T-RO 37(6):1855–1873, 2021	handover: pre-handover (giao tiếp tường minh/ngầm, căn tư thế) → physical handover (bắt đầu khi tay người nhận chạm vật)	các mốc pha	thời gian chờ, idle, tổng thời gian handover

Ánh xạ sang schema của bạn

- g_obj ≈ "noun + box" của Ego4D STA, nhưng mở rộng sang *vùng/vị trí* (vì Instructor chỉ hướng/vùng, Intruder xâm nhập vùng làm việc). Nên dùng từ điển cố định gồm K_obj đối tượng + K_reg vùng (ví dụ: vùng làm việc robot, vùng bàn giao, ngoài vùng).
- g_act ≈ verb, nhưng là *verb của người*: {take, give/handover-to-robot, request/receive-from-robot, place, point/command, pass-by, reach-into, approach, idle/none}. Nên giữ factorized (g_obj, g_act) thay vì nhãn joint: cách EPIC/Ego4D báo cáo riêng verb/noun và chấm khớp đồng thời cho thấy cách này dễ phân tích. Hơn nữa, decomposition g_obj / g_act / g_obj+g_act của bạn đòi hỏi factorization.
- τ ≈ TTC của Ego4D (đơn vị giây) cộng thêm phase progress 0→1.
- ξ ≈ "future hand movement" của Ego4D (vị trí tay tương lai), mở rộng thành keypoint 3D trên horizon H.

1.2. Định nghĩa mốc thời gian theo cách thao tác được (đây là đề xuất của tôi, ghép từ Ego4D và Ortenzi)

- `t_onset`: khung đầu tiên người bắt đầu chuyển động hướng về ý định (tương tự PRE frame / bắt đầu pre-handover).

- `t_clear`: khung sớm nhất mà ý định *không còn mơ hồ*. Định nghĩa thao tác khuyến nghị là "truncated-video test": cho k ⩾ 2 annotator xem video bị cắt tại t, rồi lấy t sớm nhất mà đa số chọn đúng (g_act, g_obj). Cách thay thế là t sớm nhất mà posterior của một classifier offline vượt ngưỡng (ví dụ 0,8) và giữ ổn định trong N khung.
- `t_event`: CONTACT/PNR (chạm vật, tay vào vùng bàn giao, tay đi qua ranh giới workspace với Intruder).
- Từ đó: TTE(t) = t_event − t; phase τ(t) = clip((t − t_onset)/(t_event − t_onset), 0, 1); thêm hai trạng thái rời rạc "chưa bắt đầu" (t < t_onset) và "đã xong" (t > t_event).
- Ô "early response" trong thiết kế 2×2: robot phản ứng trong [t_onset, t_clear); ô "clear": phản ứng sau t_clear. Lưu cả t_robot_response để tính response latency so với t_clear (âm nghĩa là có tính dự đoán).

1.3. Quy trình gán nhãn

1. Ghi đồng bộ RGB(-D) + qpos + timestamp. Chạy pose estimator (MediaPipe Hands/Pose, RTMPose qua MMPose, hoặc OpenPose), nâng lên 3D bằng depth, rồi đổi sang **khung gốc robot** bằng extrinsic calibration.
2. Gán (g_act, g_obj) theo *segment* (một ý định cho mỗi đoạn [t_onset, t_event]) rồi lan ra từng timestep. Ngoài segment thì gán "none".
3. Gán t_onset, t_event bằng công cụ có tier thời gian (ELAN tiện cho nhiều tier; CVAT/Label Studio tiện cho bbox/keypoint). t_clear lấy theo truncated-video test.
4. Đo inter-annotator agreement: Cohen/Fleiss κ cho g_act, g_obj; với mốc thời gian, dùng sai số tuyệt đối trung bình và tỉ lệ nằm trong dung sai 0,25 s (theo ngưỡng TTC của Ego4D).
5. Tính ξ: lấy keypoint (cổ tay, đầu ngón, có thể khuỷu/vai) từ t+Δ đến t+H, làm mượt (Savitzky–Golay hoặc lọc one-euro), biểu diễn *tương đối* với vị trí tay hiện tại, rồi chuẩn hóa z-score theo thống kê tập train.

Chống rò rỉ đáp án (answer leakage): trong Instructor, "point → vùng trái" gần như đồng nghĩa "robot move left". Điều này chấp nhận được vì đó vẫn là ý định của người, nhưng phải báo cáo rõ, và kiểm chứng bằng D-shuffled cùng counterfactual swap (Bước 5) để chứng minh policy *dùng* intent chứ không chỉ học ánh xạ tra bảng. Tuyệt đối không mã hóa hành động robot hay thời điểm robot cần phản ứng vào τ/ξ.

Bước 2 — Biểu diễn và encoder cho từng thành phần

Thành phần	Khuyến nghị chính	Phương án thay thế	Ghi chú
g_obj	`nn.Embedding(K_obj+1, d)`, chỉ số cuối = UNKNOWN	one-hot → Linear; hoặc token vị trí 3D của đối tượng (Linear(3,d)) cộng embedding lớp	Nếu đối tượng có vị trí thay đổi, cộng thêm token vị trí để g_obj có "grounding" không gian
g_act	`nn.Embedding(K_act+1, d)`	one-hot → Linear	Factorized; khi dùng nhãn dự đoán thì đưa *soft embedding* Σ p_k·E_k
τ	MLP([τ, log(1+TTE), sin/cos(2^k π τ) với k=0..F−1]) → d	binned: 10–20 bin + Embedding (thêm bin "chưa bắt đầu"/"đã xong")	Fourier features + MLP là cách phổ biến cho đặc trưng chuyển động (ví dụ CDKFormer dùng Fourier embedding rồi MLP); log-time giúp phân giải tốt gần sự kiện
ξ	per-waypoint token: subsample H thành M = 5–10 waypoint, mỗi waypoint Linear(J·3, d) + sinusoidal time embedding	(a) một token MLP trên vector phẳng; (b) DCT hệ số thấp (như siMLPe) → Linear; (c) encoder nhỏ kiểu Wayformer/early fusion	Khung gốc robot, tương đối với tay hiện tại, z-score

- Mỗi thành phần cũng nên có một **type embedding** riêng (Embedding(4, d)) cộng vào token, cùng với positional embedding học được (xem Bước 3), để transformer phân biệt nguồn.
- Wayformer cho thấy "early fusion", tức chiếu mọi modality về cùng chiều rồi đưa chung vào một encoder attention, là cách đơn giản và hiệu quả. alphaxiv Agent history được biểu diễn dạng [A, T, 1, D_h] với mỗi timestep là một vector trạng thái. arxiv Cách per-waypoint token của bạn chính là early fusion trong encoder ACT.

Bước 3 — Chèn vào ACT (code-level)

3.1. Những gì đã kiểm chứng trong tonyzhaozh/act

- `detr_vae.py` (commit 742c753, dòng 131, trích trong issue #25): `hs = self.transformer(src, None, self.query_embed.weight, pos, latent_input, proprio_input, self.additional_pos_embed.weight)[0]`. github
- `[0]` ở đây chỉ lấy output của lớp decoder *đầu tiên* (issue #25 đã nêu quirk này). github Khi so sánh, hãy giữ nguyên hành vi này cho cả baseline A và các biến thể D.
- Các dòng dưới đây **dựa trên trí nhớ, cần đối chiếu với file trước khi trích dẫn trong đồ án:** `self.additional_pos_embed = nn.Embedding(2, hidden_dim)` (proprio + latent); `self.register_buffer('pos_table',`

`get_sinusoid_encoding_table(1+1+num_queries, hidden_dim))` cho CVAE encoder ([CLS], qpos, a_seq); `encoder_input = torch.cat([cls_embed, qpos_embed, action_embed], axis=1)`; trong `Transformer.forward`: `addition_input = torch.stack([latent_input, proprio_input], axis=0)`; `src = torch.cat([addition_input, src], axis=0)`; `pos_embed = torch.cat([additional_pos_embed, pos_embed], axis=0)`.

3.2. Tiền lệ thực tế: P+ACT (github.com/Likerener/prox_learning) Repo này thêm token proximity vào ACT đúng theo cách bạn định làm: thêm `input_proj_proximity = Linear(3, hidden_dim)`, mở rộng `additional_pos_embed` thành `(2 + n_proximity_sensors·K, hidden_dim)`, trong `transformer.py` "concatenates the prox tokens after [latent, proprio] and before image tokens", và truyền tham số mới qua `ACTPolicy.__call__` và `detr/main.py`. github Bạn có thể sao chép cấu trúc diff này.

3.3. Phác thảo sửa đổi (pseudo-PyTorch, minh họa)

python

```
# detr_vae.py — __init__
self.intent_enc = IntentEncoder(d=hidden_dim, n_obj=K_obj, n_act=K_act, M=M_way
self.n_intent = 1 + 1 + 1 + M_waypoints          # g_obj, g_act, tau, xi(M)
self.additional_pos_embed = nn.Embedding(2 + self.n_intent, hidden_dim)
# CVAE encoder: [CLS], qpos, intent tokens, action seq
self.register_buffer('pos_table',
    get_sinusoid_encoding_table(1 + 1 + self.n_intent + num_queries, hidden_dim

# forward (training)
intent_tok = self.intent_enc(intent, drop_mask)   # (B, n_intent, d), đã thay b
encoder_input = torch.cat([cls_embed, qpos_embed, intent_tok, action_embed], ax
...
hs = self.transformer(src, None, self.query_embed.weight, pos,
                      latent_input, proprio_input, self.additional_pos_embed.we
                      extra_input=intent_tok.permute(1, 0, 2))[0]

# transformer.py — Transformer.forward
addition_input = torch.cat([torch.stack([latent_input, proprio_input], 0), extr
src = torch.cat([addition_input, src], axis=0)
pos_embed = torch.cat([additional_pos_embed, pos_embed], axis=0)  # additional_
```

- **CVAE encoder**: đưa *cùng* intent token (sau dropout) vào encoder. Khi đó z chỉ cần mã hóa phần "style" còn lại. Nhưng ở những mẫu intent bị drop, z có thể hấp thụ intent từ chuỗi hành động. Không có rò rỉ lúc test (vì z = 0), nhưng có thể làm decoder lười đọc ảnh khi training. Cần đo bằng **probe**: huấn luyện linear classifier từ μ(z) → g_act/g_obj. Nếu độ chính xác vượt xa mức ngẫu nhiên, hãy tăng kl_weight hoặc tắt intent-dropout trong nhánh encoder.
- Một preprint mới, "The Latent That Never Was" (Bo Kang, Ghent Univ., arXiv 2609.16745, 15/09/2026), chạy lại ablation CVAE của ACT. Kết quả: latent hầu như không cải thiện reconstruction trên các task ACT ở các KL weight dương đã thử, và theo

bài, "Its encoder accounts for a third of the parameters in the tested single-camera LeRobot model". Bài ACT gốc báo cáo success giảm từ 35,3% xuống 2% khi bỏ encoder, nhưng lần chạy lại 10.000 bước ra kết quả ngược lại: 26,0% khi có encoder và 45,2% khi không có. Nếu đúng, rủi ro "z hấp thụ intent" có thể nhỏ, nhưng vẫn nên đo bằng probe thay vì giả định.

3.4. LeRobot ACT (`src/lerobot/policies/act/modeling_act.py`)

- Encoder input là danh sách token 1D [latent, robot_state, (env_state)] + token ảnh. Positional embedding 1D là `encoder_1d_feature_pos_embed = nn.Embedding(n_1d_tokens, dim_model)`, còn ảnh dùng `ACTSinusoidalPositionEmbedding2d`. PR #314 (thêm `observation.environment_state`) là mẫu sửa đổi tốt nhất: tăng `n_1d_tokens` và thêm `encoder_env_state_input_proj`. github deepwiki Hãy làm y hệt với `observation.intent.*`.
- Mặc định LeRobot: chunk_size 100, n_action_steps 100, use_vae True, kl_weight 10, latent_dim 32. Lúc inference, latent = 0. deepwiki
- Tên biến của nhánh VAE encoder trong LeRobot chưa được kiểm chứng trong nghiên cứu này, cần đọc trực tiếp file.

3.5. Phương án dự phòng FiLM

- **MT-ACT (RoboAgent; Bharadhwaj, Vakil, Sharma, Gupta, Tulsiani, Kumar; arXiv 2309.01918; ICRA 2024)**: FiLM cho language embedding để "image tokens are able to reliably focus on the language instruction". Ablation ở §5.2 của bài: "we observe a 5-10% drop in performance of the version of MT-ACT without FiLM conditioning" (phiên bản chỉ ghép token). Về chunk, §5.2 viết "a chunk size of 20 performs the best": chunk 10 giảm 0–5%, chunk 40 giảm trên 20%. Lưu ý Bảng 3 của bài ghi batch size 8, khác với batch 64 trong lệnh README. Dữ liệu 7.500 quỹ đạo, 12 kỹ năng, 38 task, 4 camera. Repo robopen/roboagent: `--chunk_size 20 --batch_size 64 --kl_weight 10 --hidden_dim 512 --dim_feedforward 3200 --lr 1e-5 --temporal_agg --multi_task`, task embedding nằm trong `constants.py`. **Chưa kiểm chứng**: FiLM gắn vào lớp ResNet nào, γ/β sinh ở đâu, và language có đồng thời là token encoder hay không. Hình 5 và §3.3 của bài liệt kê language embedding như đầu vào của transformer encoder, nên có thể là cả hai.
- **RT-1**: FiLM *identity-initialized* trong EfficientNet-B3 (tham số affine khởi tạo 0 để không phá trọng số pretrained), 81 token → TokenLearner → 8 token. arxiv github
- **Diffusion Policy**: `ConditionalResidualBlock1D` sinh `scale, shift` từ `cond_encoder(cond)` và áp `out = scale * out + shift` trong *mọi* block. github Điều kiện toàn cục `global_cond` = đặc trưng quan sát phẳng hóa. cnblogs
- **BAKU (NeurIPS 2024, arXiv 2406.07539)** cũng dùng FiLM cho task conditioning. Trên 129 task mô phỏng thuộc LIBERO, Meta-World và DeepMind Control Suite, BAKU báo cáo "an overall 18% absolute improvement over RT-1 and MT-ACT", và cải thiện 36% trên LIBERO khó hơn.
- Một preprint (arXiv 2609.20669, "Learning Foresight without Explicit Trajectories for 3D Diffusion Policies") báo cáo gated FiLM chỉ ở bottleneck đạt 62,8%, so với 55,8% (FiLM mọi block) và 52,9% (cross-attention). Các con số này chỉ có từ snippet tìm kiếm,

vì PDF không trích được văn bản để kiểm chứng. Gợi ý: khi dùng FiLM cho intent, nên chèn "bảo thủ", tức γ/β zero-init kèm gate.

- Đề xuất cụ thể: FiLM từ vector intent (concat các embedding) → γ, β cho output `layer3`/`layer4` của ResNet18 (zero-init), hoặc áp FiLM lên toàn bộ token của encoder transformer sau mỗi lớp.

Bước 4 — Component dropout

- **Tiền lệ**: Octo "randomly zero out the language instruction or goal image per training example" và mask hoàn toàn token không tồn tại. arxiv arxiv BESO (Reuss et al., KIT, arXiv 2304.02532) dùng goal dropout 0,1 để học đồng thời π(a|s,g) và π(a|s). Ablation của BESO cũng cho thấy FiLM kém hơn ghép tuần tự một chút: 0,93→0,91 trên Block-Push và 3,76→3,4 trên Kitchen. CFG nói chung thay điều kiện bằng *learned null embedding* với xác suất p_drop. arxiv
- **Khuyến nghị**:
  - Mỗi thành phần c ∈ {g_obj, g_act, τ, ξ} bị drop *độc lập* với p_c ≈ 0,15–0,3, thay bằng **null token học được riêng cho từng thành phần** (`nn.Parameter` cho τ, ξ; chỉ số UNKNOWN cho Embedding).
  - Thêm xác suất drop-all p_all ≈ 0,1 để model con "không intent" xấp xỉ baseline A.
  - Với ξ, có thể drop theo khối thời gian (ẩn các waypoint xa) để mô phỏng forecast ngắn.
- **Null token hay zero hay mask?** Zero vector cộng positional/type embedding vẫn là một token "có nghĩa", nên null token học được rõ ràng hơn. Mask bằng `src_key_padding_mask` (giống Octo) tương đương "vắng mặt" thật, và có thể dùng vì `Transformer.forward` của ACT nhận tham số `mask`. Nên báo cáo một ablation nhỏ giữa null token và mask.
- **Đánh giá subset**: ở test, giữ thành phần ∈ S và thay phần còn lại bằng null. Lưu ý: model dropout đánh giá theo subset *không tương đương* các model huấn luyện riêng. Hãy huấn luyện thêm 1–2 model chuyên biệt (ví dụ chỉ g_act, và full) để kiểm tra chéo kết luận của decomposition.
- Nếu muốn kiểu "guidance": với head phân loại 9 hành động, có thể trộn logits $\ell = \ell_\varnothing + \lambda(\ell_S - \ell_\varnothing)$ (tương tự CFG). BESO §IV ghi "In diffusion literature, λ commonly ranges from 2 to 7.5", nhưng với policy thì giá trị λ tốt nhất là 1,25 trên hầu hết môi trường thử, nên hãy bắt đầu với λ gần 1. Đây là mở rộng tùy chọn, chưa có tiền lệ cho ACT.

Bước 5 — Giao thức đánh giá

5.1. Ba tầng thí nghiệm

1. *D-full-oracle vs A*: giá trị của thông tin hoàn hảo, báo cáo theo vai (Instructor/Collaborator/Intruder) × ô 2×2.
2. *Decomposition lồng nhau*: g_obj, g_act, g_obj+g_act, +τ, +ξ, dùng cùng model dropout, cộng kiểm tra chéo bằng model chuyên biệt.
3. *Predicted intent*: thay oracle bằng predictor cho subset tốt nhất, và báo cáo "khoảng cách oracle–predicted".

5.2. Metric

- Mức tương tác (theo HRIBench): CSR, CT, Idle Ratio, TSync, Response Latency (so với t_clear), Order Compliance, CFR/HCS cho Intruder. (arxiv)
- Mức hành động: độ chính xác 9 hành động rời rạc theo từng timestep, **false-start rate** (hành động hỗ trợ trước t_onset, hoặc sai g_act trước t_clear), như cảnh báo trong Learning to Assist.
- Khuyến nghị kỹ thuật: vì action space là 9 lớp rời rạc, nên thêm head phân loại (cross-entropy) cho mỗi phần tử chunk thay cho L1. Điều này cho log-likelihood để ước lượng CMI. Đây là sai khác so với ACT gốc vốn dùng L1 trên hành động liên tục, cần ghi rõ.

5.3. Manipulation checks

- *Intent change rate*: tỉ lệ timestep mà hành động argmax thay đổi khi swap/zero/shuffle một thành phần.
- *Counterfactual grounding rate*: trong các lần swap g → g', tỉ lệ hành động đổi thành hành động *nhất quán với g'* (cần bảng ánh xạ chuẩn intent → hành động đúng, do người thiết kế kịch bản lập).
- *CMI*: $\hat{I}(A; C \mid O, C_{rest}) \approx E[\log p(a \mid o, c, c_{rest}) - \log p(a \mid o, \varnothing_c, c_{rest})]$, tính trên tập test bằng head phân loại.
- *Controls*: D-shuffled (intent xáo trộn giữa episode: nếu vẫn tốt bằng D thì intent không được dùng), D-aux (intent chỉ là loss phụ), A-mask/D-mask (che vùng người: D-mask tốt và A-mask kém nghĩa là intent thay thế được thông tin thị giác về người).

5.4. Tính hợp lệ của oracle

- Oracle τ/ξ chỉ hợp lệ khi chuyển động người cố định, không phụ thuộc robot: offline (open-loop trên log), hoặc sim với chuyển động scripted/replay, như HRIBench cố định lịch perturbation giữa các model. (arxiv)
- Với người thật phản ứng, tương lai của người phụ thuộc hành động robot (đây là động lực của InteRACT), nên chỉ dùng predicted intent. Báo cáo kết quả closed-loop thật như một nghiên cứu khả thi nhỏ, tách khỏi kết luận oracle.

Bước 6 — Predictor cho bước "predicted intent"

Thành phần	Predictor gợi ý	Chi tiết đã kiểm chứng
g_act, g_obj	Classifier chuỗi keypoint tay/thân (LSTM/Transformer); thêm gaze nếu có	Một nghiên cứu HRC (PMC12757248) phân loại quỹ đạo tay theo thời gian thực, so sánh LSTM và transformer; có công trình riêng về nhận diện ý định qua gaze (arXiv 2405.07570)
τ (TTE)	Hồi quy smooth-L1 từ cùng encoder	Trong Ego4D STA, SOIA-DOD dùng smooth L1 cho TTC; TAP-JEPA lấy mẫu anticipation time 0,25–1,75 s khi train để tăng cường
ξ	siMLPe (DCT + MLP), fine-tune trên dữ liệu của bạn	siMLPe: MPJPE trên Human3.6M 9,6 / 21,7 / 46,3 / 57,3 / 75,7 / 90,1 / 101,8 / 109,4 mm tại 80–1000 ms, chỉ 0,14M tham số (Guo et al., WACV 2023; 0,138M, 48 khối MLP theo Bảng 5)
ξ (hướng HRC)	ManiCast	Pretrain AMASS, fine-tune CoMaD (mocap), upsample điểm chuyển tiếp và tăng trọng số khớp quan trọng với cost của planner; dùng với MPC (mlr) (github)
Intent có điều kiện theo robot	InteRACT (Kedia, Bhardwaj, Dan, Choudhury; ICRA 2024, tr. 621–628; arXiv 2311.12943)	Hành động tương lai của robot làm query cho decoder để dự đoán z_H; pretrain human–human, fine-tune human–robot

- **Huấn luyện policy với intent dự đoán**: dùng *cross-fitting* (k-fold: predictor dự đoán trên fold không được train) để policy thấy lỗi dự đoán thực tế. Đưa *xác suất mềm* thay vì argmax, và có thể trộn oracle/predicted theo lịch (oracle nhiều ở đầu, predicted nhiều về cuối).

Recommendations

1. **Thứ tự triển khai**: (a) pipeline nhãn + IAA trên 10–20% dữ liệu → (b) ACT + token g_obj/g_act (sửa theo mẫu P+ACT) → (c) thêm τ, ξ, null token, dropout → (d) manipulation check → (e) FiLM chỉ khi intent change rate thấp dù CMI của dữ liệu cao.
2. **Siêu tham số khởi đầu**: hidden 512, ff 3200, enc 4/dec 7, heads 8, lr 1e-5, kl 10 (giữ như ACT để A so sánh công bằng). **chunk_size 20–50** thay vì 100, vì MT-ACT thấy 20 tốt nhất và Learning to Assist cho thấy horizon dài làm tăng false start. (github +2) Thử temporal ensembling. p_c = 0,2, p_all = 0,1, M = 8 waypoint, H ≈ 1–1,5 s, F = 6 tần số Fourier cho τ.
3. **Định nghĩa trước (pre-register)** bảng intent → hành động đúng cho counterfactual grounding, cùng ngưỡng t_clear, trước khi xem kết quả.
4. **Báo cáo theo vai**: kỳ vọng lợi ích của intent lớn nhất ở Collaborator/Intruder (nơi HRIBench cho thấy policy yếu nhất về thời điểm và an toàn), còn ở Instructor thì cần chứng minh không phải leakage. (arxiv)

Caveats

- HRIBench (2607.13056), Learning to Assist (2606.12475), "The Latent That Never Was" (2609.16745) và preprint gated FiLM (2609.20669) đều là **preprint 2026 chưa qua bình duyệt**. Con số của chúng có thể thay đổi.
- Nhiều dòng code ACT trong Bước 3.1 được nêu từ trí nhớ (đã đánh dấu). Chi tiết vị trí FiLM của MT-ACT chưa được xác minh từ code. Hãy mở trực tiếp `detr/models/detr_vae.py`, `transformer.py` (tonyzhaozh/act) và `detr/models/backbone.py`, `constants.py` (robopen/roboagent) trước khi trích dẫn.
- Các công cụ MediaPipe/RTMPose/OpenPose, ELAN/CVAT/Label Studio được đề xuất dựa trên hiểu biết chung, chưa đối chiếu tài liệu chính thức trong nghiên cứu này. Các giá trị p_c, M, H, F là điểm khởi đầu hợp lý chứ không phải giá trị đã được công bố cho bài toán này.
- Kết luận từ model dropout đánh giá theo subset có thể khác model huấn luyện riêng. CMI ước lượng qua head phân loại chỉ là xấp xỉ, phụ thuộc hiệu chỉnh (calibration) của model.